

© The Author(s), under exclusive license to APress Media, LLC, part of Springer Nature 2022

V. Sarcar, *Java Design Patterns*

https://doi.org/10.1007/978-1-4842-7971-7_19

19. Command Pattern

Vaskaran Sarcar¹

(1) Garia, Kolkata, India

This chapter covers the Command pattern.

GoF Definition

It encapsulates a request as an object, thereby letting you parameterize clients with different requests, queue or log requests, and support undoable operations.

Concept

Using this pattern, you encapsulate a method invocation process. Here an object can invoke an operation through some crystalized method, and it does not worry about how to perform the intended operation. This pattern is often used in a multithreaded environment. It's one of those patterns that is normally tough to understand by merely reading the description. The concept becomes clearer when you see the implementations. So, stay with me and keep reading until you see demonstration 1.

In general, four terms are important here: *invoker*, *client*, *command*, and *receiver*, which are as follows:

- The `command` object consists of the action(s) that a `receiver` will perform.
- A `command` object can invoke a method of the `receiver` in a way that is specific to that receiver's class. The `receiver` then starts processing the job (or the action).
- A `command` object is separately passed to the `invoker` object to invoke the command. The `invoker` object contains those crystallized methods through which a client can perform a job without worrying about how the actual job is performed by the target receiver.
- The `client` object holds the `invoker` object and the `command` objects. The client must decide which commands to execute before it passes the appropriate command to the `invoker` object.

Real-Life Example

When you draw a picture, you may need to redraw (undo) some parts of it to make it better.

Computer World Example

In general, you can observe this pattern in the menu system of an editor or integrated development environment (IDE). For example, you can use the Command pattern to support an Undo operation or the Undo all previous operations in a software application. Here is another common example:

- When you use the `java.lang.Runnable` interface and use the `run()` method to invoke the target receiver's method, you use Command design pattern.

It is interesting to note that Microsoft also used this pattern in Windows Presentation Foundation (WPF). The article named "The Command Pattern in .NET" from the Visual Studio Magazine site (<https://visualstudiomagazine.com/articles/2012/04/10/command-pattern-in-net.aspx>) describes it in detail:

"The command pattern is well suited for handling GUI interactions. It works so well that Microsoft has integrated it tightly into the Windows Presentation Foundation (WPF) stack. The most important piece is the `ICommand` interface from the `System.Windows.Input` namespace. Any class that implements the `ICommand` interface can be used to handle a keyboard or mouse event through the common WPF controls. This linking can be done either in XAML or in a code-behind."

Note Implementations for undos vary and can be complex. The Memento design pattern (covered in Chapter 20) also supports undo operations and you may need to use both design patterns in your application to implement a complex undo operation.

Implementation

Game is the receiver class in the upcoming example. It has three methods and one constructor. It has the following definition:

```
class Game {
    String gameName;
    public Game(String name) {
        this.gameName = name;
    }
    public void start() {
        System.out.println(gameName + " is on.");
    }
}
```

```

    }
    public void displayScore() {
        System.out.println("The score is changing from time to time.");
    }
    public void stop() {
        System.out.println("The game of " + gameName + " is stopped.");
    }
}

```

GameStartCommand and GameStopCommand are two concrete classes to represent commands. These two classes implement the common interface Command, which is as follows (the associated comments tell the purposes of these methods):

```

interface Command {
    // To execute a command
    void executeCommand();
    // To undo last command execution
    void undoPreviousCommand();
}

```

First, look at the concrete command class named GameStopClass. Notice that the concrete command class stores the receiver (Game) as an instance variable. It also uses the receiver's actions inside its method bodies.

```

class GameStopCommand implements Command {
    private Game game;
    public GameStopCommand(Game game) {
        this.game = game;
    }
    @Override
    public void executeCommand() {
        System.out.println("Finishing the game.");
        game.stop();
    }
    @Override
    public void undoPreviousCommand() {
        System.out.println("Undoing the stop command.");
        game.start();
        game.displayScore();
    }
}

```

Here RemoteControlInvoker is the invoker class.

Note I append the word “Invoker” for your easy understanding. I agree that RemoteControl can be a better choice when you write real-world code.

It has the following definition:

```
class RemoteControlInvoker {
    Command currentCommand, lastCommandPerformed;
    public void setCommand(Command command) {
        this.currentCommand = command;
    }
    public void executeSelectedCommand() {
        currentCommand.executeCommand();
        lastCommandPerformed = currentCommand;
    }
    public void undoCommand() {
        // Undo the last command executed
        lastCommandPerformed.undoPreviousCommand();
    }
}
```

How can a client use this pattern? At a high level, there are the following steps:

- **Step 1:** A client uses a receiver object to create a command object.

```
Game gameName = new Game("Golf");
// Command to start the game
GameStartCommand gameStartCommand = new
    GameStartCommand(gameName);
```

- **Step 2:** Next, it sets the command to an invoker and uses the `executeSelectedCommand()` method to execute the command.

```
System.out.println("Starting the game using a command.");
invoker.setCommand(gameStartCommand);
invoker.executeSelectedCommand();
```

Now investigate the method body of the `executeSelectedCommand()` method, which is as follows:

```
currentCommand.executeCommand();
// Remaining code skipped
```

It means that the call is delegated to the target receiver's (Game class object in this example) `executeCommand()`. I kept some console messages for your easy reference.

One last point: notice the `executeMethod()` of `GameStartClass`:

```
public void executeCommand() {
    System.out.println("Game is about to start.");
    game.start();
    game.displayScore();
}
```

```
}

```

You can see that a particular command doesn't restrict itself to perform a single action only; instead, based on your needs, you can perform a series of actions on a target receiver and encapsulate them in a command object.

Points To Note

The examples in this chapter show simple demonstrations of undo operations. The implementation of an undo depends on the specification and can be complex in some scenarios. For demonstration 1, an undo call simply undoes the last command that was performed successfully. If you notice the `executeCommand()` and `undoPreviousCommand()` methods of the `GameStartCommand` and `GameStopCommand` classes, you will see that they are just doing the opposite, so when a client invokes an undo operation using `GameStopCommand`, the game will start again and display the score details (which is a simple console message in this example), but if the client invokes the undo operation using `GameStartCommand`, the game will finish immediately. It is similar to switching on a light and switching off the same light, or adding a number to a target number and, as a reverse case, subtracting the same number from the resultant number again.

Class Diagram

Figure 19-1 shows the class diagram.

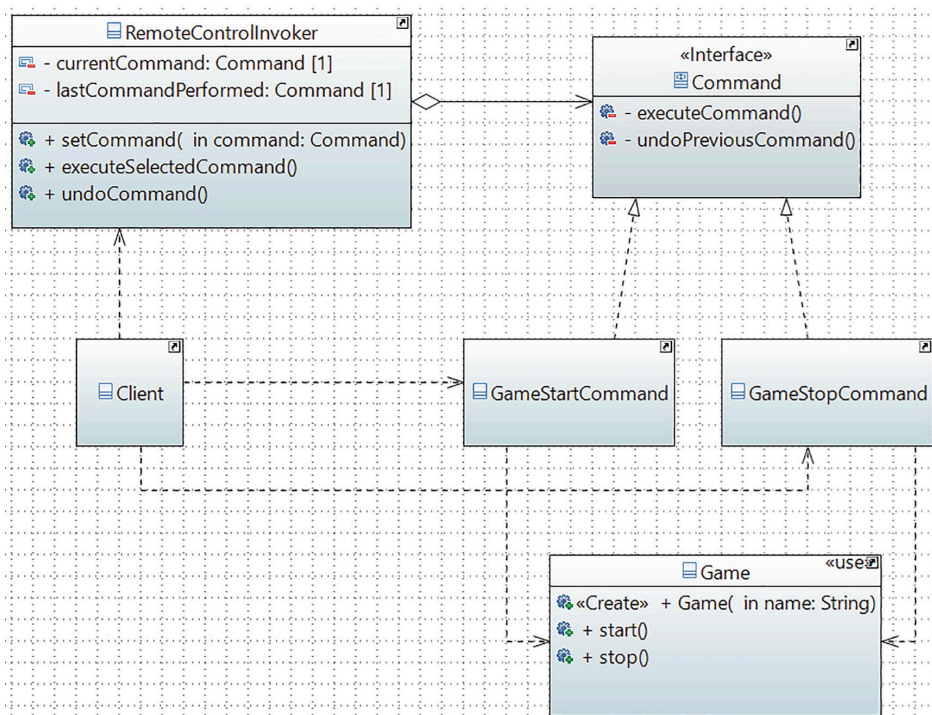


Figure 19-1 Class diagram

Package Explorer View

Figure **19-2** shows the high-level structure of the program. To make the diagram short, I did not expand `GameStopCommand`, which has a similar structure as `GameStartCommand`. (The `implementation_2` folder is used for the next demonstration).

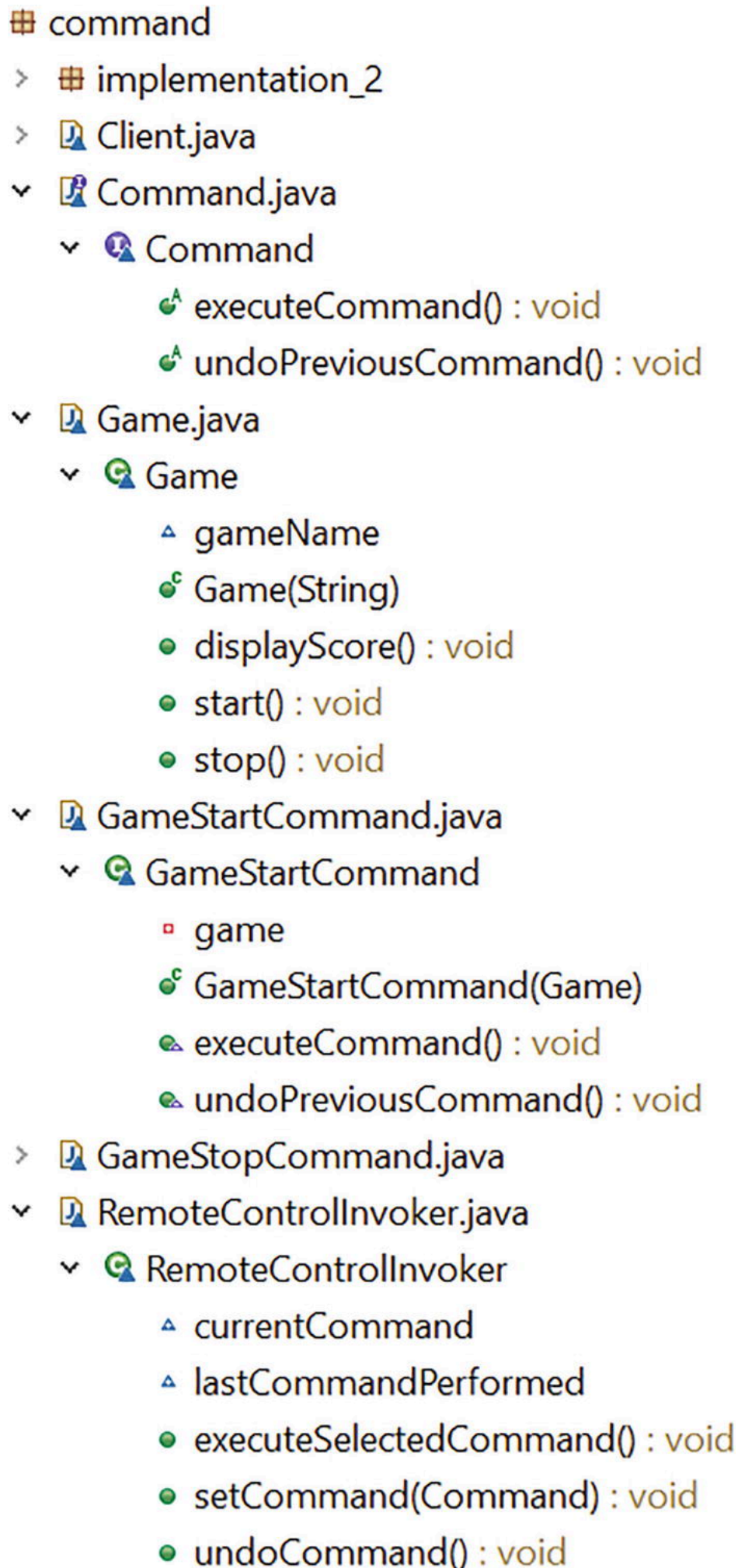


Figure 19-2 Package Explorer view

Demonstration 1

Here's the complete program. All the classes and interfaces are stored in the package `jdp3e.command`.

```
// Game.java
class Game {
    String gameName;
    public Game(String name) {
        this.gameName = name;
    }
    public void start() {
        System.out.println(gameName + " is on.");
    }
    public void displayScore() {
        System.out.println("The score is changing from time to time.");
    }
    public void stop() {
        System.out.println("The game of " + gameName + " is stopped.");
    }
}

// Command.java
interface Command {
    // To execute a command
    void executeCommand();
    // To undo last command execution
    void undoPreviousCommand();
}

// GameStartCommand.java
class GameStartCommand implements Command {
    private Game game;
    public GameStartCommand(Game game) {
        this.game = game;
    }
    @Override
    public void executeCommand() {
        System.out.println("The game is about to start.");
        game.start();
        game.displayScore();
    }
    @Override
    public void undoPreviousCommand() {
        System.out.println("Undoing the start command.");
        game.stop();
    }
}

// GameStopCommand.java
class GameStopCommand implements Command {
    private Game game;
    public GameStopCommand(Game game) {
        this.game = game;
    }
}
```

```

    }
    @Override
    public void executeCommand() {
        System.out.println("Finishing the game.");
        game.stop();
    }
    @Override
    public void undoPreviousCommand() {
        System.out.println("Undoing the stop command.");
        game.start();
        game.displayScore();
    }
}

// RemoteControlInvoker.java (Invoker)
class RemoteControlInvoker {
    Command currentCommand, lastCommandPerformed;
    public void setCommand(Command command) {
        this.currentCommand = command;
    }
    public void executeSelectedCommand() {
        currentCommand.executeCommand();
        lastCommandPerformed = currentCommand;
    }
    public void undoCommand() {
        // Undo the last command executed
        lastCommandPerformed.undoPreviousCommand();
    }
}

// Client.java
class Client {
    public static void main(String[] args) {
        System.out.println("***Command Pattern Demonstration.***\n");
        // Receiver
        Game gameName = new Game("Golf");
        // Command objects:
        // Command to start the game
        GameStartCommand gameStartCommand = new
            GameStartCommand(gameName);
        // Command to stop the game
        GameStopCommand gameStopCommand = new
            GameStopCommand(gameName);

        // Invoker
        RemoteControlInvoker invoker = new
            RemoteControlInvoker();
        System.out.println("---Testing Scenario-1---\n");
        System.out.println("Starting the game using a command.");
        invoker.setCommand(gameStartCommand);
        invoker.executeSelectedCommand();
        System.out.println("\nUndoing the previous command now.");
        invoker.undoCommand();
        System.out.println("\n---Testing Scenario-2---\n");
        System.out.println("Starting the game again.");
    }
}

```

```

        invoker.setCommand(gameStartCommand);
        invoker.executeSelectedCommand();
        System.out.println("\nIssuing a stop command now.");
        invoker.setCommand(gameStopCommand);
        invoker.executeSelectedCommand();
        System.out.println("\nUndoing the previous command now.");
        invoker.undoCommand();
    }
}

```

Output

Here's the output:

```

***Command Pattern Demonstration.***
---Testing Scenario-1---
Starting the game using a command.
The game is about to start.
Golf is on.
The score is changing from time to time.
Undoing the previous command now.
Undoing the start command.
The game of Golf is stopped.
---Testing Scenario-2---
Starting the game again.
The game is about to start.
Golf is on.
The score is changing from time to time.
Issuing a stop command now.
Finishing the game.
The game of Golf is stopped.
Undoing the previous command now.
Undoing the stop command.
Golf is on.
The score is changing from time to time.

```

Q&A Session

19.1 The GoF definition starts with “Encapsulate a request.” How are you implementing the encapsulation in demonstration 1?

Notice that the command object contains the set of actions that target a specific receiver. When you set the command and invoke the `executeSelectedCommand()` on the invoker object, the intended actions are performed at the receiver's end. From the invoker's point of view, it simply knows that if it calls the `executeSelectedCommand()`, the requests will

be processed by the target receiver. This method encapsulates the actions needed to get the final result/outcome from a target object/receiver.

In the next demonstration, you will also see that an invoker can handle different types of objects that can be placed in different inheritance hierarchies. For example, in this implementation, notice the use of commands to start a game or stop a game. How does this work? You set commands to start or stop a game first so that the invoker can handle the game. Let's assume you want to handle a number generator in a similar way, which means you tell the invoker to handle a number generator to increase or decrease a number. It means that you want an invoker to handle a different type of object without knowing about the target receiver. The Command pattern allows you to implement a similar type of implementation. In this case, at the top level, from an invoker perspective, it simply invokes the corresponding methods of the target receiver(s).

19.2 Following the GoF definition, how did you parameterize other objects with different requests?

Notice the following method in the `RemoteControlInvoker` class:

```
public void setCommand(Command command) {  
    this.currentCommand = command;  
}
```

You can see that this method can be parameterized with different commands. Here is a partial segment of testing scenario-2 inside the client code, which is as follows:

```
System.out.println("Starting the game again.");  
invoker.setCommand(gameStartCommand);  
invoker.executeSelectedCommand();  
System.out.println("\nIssuing a stop command now.");  
invoker.setCommand(gameStopCommand);  
invoker.executeSelectedCommand();
```

Notice that you first set `GameStartCommand` in the invoker and later replace it with `GameStopCommand`. The invoker object invokes the `executeSelectedCommand()` in both cases. It is possible because the invoker knows that all these commands can be executed using this method.

19.3 In this example, you are dealing with a single receiver only. How can you deal with multiple receivers?

No one restricts you from creating a new class and following the implementation that is shown in demonstration 1. In demonstration 2, you'll

see such an implementation.

But before I show you demonstration 2, I want you to notice another point in this context. In demonstration 1, you create a `Game` class object using the following line:

```
Game gameName = new Game("Golf");
```

Since the `Game` class constructor accepts a `String` parameter, you could also pass a different value and create a different `Game` object. For example, if you append the following code segment inside the client code

```
System.out.println("\n-----Testing Scenario-3-----");
System.out.println("Playing a different game now.\n");
// Receiver
gameName = new Game("Soccer");
// Command to start the game
gameStartCommand = new GameStartCommand(gameName);
// Command to stop the game
gameStopCommand = new GameStopCommand(gameName);
System.out.println("Starting the game using a command.");
invoker.setCommand(gameStartCommand);
invoker.executeSelectedCommand();
System.out.println("\nUndoing the previous command now.");
invoker.undoCommand();
```

you get the following output:

```
-----Testing Scenario-3-----
Playing a different game now.
Starting the game using a command.
The game is about to start.
Soccer is on.
The score is changing from time to time.
Undoing the previous command now.
Undoing the start command.
The game of Soccer is stopped.
```

19.4 Can I ignore the use of the invoker object?

The invoker plays an important role. Its usage is explained in Q&A 19.1. Most of the time, programmers try to encapsulate data and the corresponding methods in object-oriented programming. But using the Command pattern, you try to encapsulate command objects. In other words, you implement encapsulation from a different perspective.

I told you earlier that when `executeSelectedCommand()` of the invoker object is called, the intended actions are performed at the receiver's end.

From the outside, no other objects have a clue about how it happens; they simply know that if they call this method, their requests will be processed. So, simply put, an invoker contains some crystalized method through which a client can perform a job without worrying how the actual job is performed at the receiver's end. This approach makes sense when you deal with a complex set of commands.

Let's review the terms again. To invoke a receiver's method, you create command objects. You execute those commands through an invoker that calls the actual methods of the receiver. Using this approach, you decouple the caller and the receiver. So, the invoker is actually the middleman who helps you implement a nice object-oriented design. If your program is very short and simple, you may feel that this invoker class is not mandatory. For example, consider a case in which a command object has only one method to execute and so you try to dispense with an invoker to invoke the method. But invokers may play an important role when you want to handle different objects that reside in different inheritance hierarchies. It also helps you keep track of a series of commands in a log file (or in a queue).

19.5 Why would you want to keep track of these logs?

You may want to create undo or redo operations.

19.6 What are the key advantages associated with the Command pattern?

Here are some advantages:

- The Command pattern decouples the sender and receiver objects. In other words, it helps you decouple an object making a request from an object that receives the request and performs the intended action. Notice that an invoker can work with a receiver without directly knowing about it. In addition, a client also does not know how a middleman (invoker) fulfils its needs.
- An invoker can handle different types of objects that perform different types of works. Demonstration 2 will make this point clear to you.
- You can create macro commands (these are sequences of multiple commands and can be invoked together. Here's an example: for a macro command, you can create a class that has a constructor to accept a list of commands. And in its `executeCommand()` method, you can invoke `executeCommand()` of these commands sequentially using a loop.
- New commands can be added without affecting the existing system.
- Most importantly, you can support the undo (and redo) operations.

- It should be noted that once you create a command object, it does not mean that the computation starts immediately. You can schedule it for later or place it in a job queue and execute it later.

19.7 What are the challenges associated with the Command pattern?

Here are some disadvantages:

- To support more commands, you need to create more classes. So, maintenance can be difficult as time goes on.
- Handling errors or making a decision about what to do with return values when an erroneous situation occurs becomes tricky. A client may want to know about them. But since you decouple the caller from a target receiver, this kind of situation is difficult to handle. The challenge becomes significant in a multithreaded environment where an invoker can run on a different thread.

19.8 In demonstration 1, you undo only the last command. Is there any way to implement undo on all of the previous commands? Also, how can you log requests?

Good question. You can simply maintain a stack that can store the commands, and then you can pop the items from the stack and invoke the `undo()` method. In Chapter [20](#) on the Memento pattern, which is similar to this pattern, I discuss more about undos and I show various implementations. For now, let me show you a simple example in which you can undo all the previous commands. Demonstration 2 is made for that. It's a simple extension of demonstration 1, so I show you the high-level class diagram. I omit the Package Explorer view, which is straightforward. You can directly jump into the implementation.

You asked another question on how to log the requests. In demonstration 2, when you maintain a list to store the commands that execute, you use this list to support undo all commands using a single method invocation. The same list can serve as a history of commands, which you can print in your target location. For example, you can make a separate file to maintain the details each time a command executes. Later you can retrieve the file for a detailed look if necessary.

Modified Implementation

This example shows a way to invoke multiple undo operations. There is no change in the Game class hierarchy, but some small changes are made to the invoker class. Here are some important points:

- Inside the invoker class, you maintain an `ArrayList` object(`savedCommands`) to store all the commands that you execute. Whenever a command is executed, it is added to this list.
 - In this example, once you call an undo operation, you remove it from the saved command list too.
 - Here you also see a method called `undoAllStoredCommand()` that is used to iterate over the `savedCommand` list and call the corresponding undo operations.
 - Finally, you clear the stored commands from this list.
- The invoker is shown with key changes in bold as follows:

```
import java.util.ArrayList;
class RemoteControlInvoker {
    Command currentCommand, lastCommandPerformed;
    ArrayList<Command> savedCommands = new
        ArrayList<Command>();
    public void setCommand(Command command) {
        this.currentCommand = command;
    }
    public void executeCommand() {
        currentCommand.executeCommand();
        savedCommands.add(currentCommand);
        lastCommandPerformed = currentCommand;
    }
    public void undoCommand() {
        // Undo the last command executed
        lastCommandPerformed.undoPreviousCommand();
        // Remove it from saved command list
        savedCommands.remove(lastCommandPerformed);
    }
    public void undoAllStoredCommands() {
        for (int i = savedCommands.size(); i > 0; i--) {
            // Get a restore point and undo previous commands
            (savedCommands.get(i - 1)).undoPreviousCommand();
        }
        // Removing everything from the
        // saved list now.
        savedCommands.clear();
    }
}
```

The `NumberGenerator` class represents the new receiver class. Note the following assumptions:

- When you initialize an instance of this class, you start with the number 0.
- The `incrementNumber()` method increments a number by 1 and the `decrementNumber()` method does the opposite. But you cannot go

beyond 0.

This class has the following definition:

```
// Receiver-2
class NumberGenerator {
    private int currentNumber;
    public NumberGenerator() {
        currentNumber = 0;
        System.out.println("The initial number is: " + currentNumber);
        System.out.println("You can increase it, but you
                                cannot decrease it further.");
    }
    public int getMyNumber() {
        return currentNumber;
    }
    public void setMyNumber(int currentNumber) {
        this.currentNumber = currentNumber;
    }
    public void incrementNumber() {
        System.out.println(" Received an
                            increment number request.");
        int presentNumber=getMyNumber();
        setMyNumber(currentNumber + 1);
        System.out.println(presentNumber + " + 1 = " + getMyNumber());
    }
    public void decrementNumber() {
        System.out.println(" Received a decrement number request.");
        int presentNumber = this.currentNumber;
        // We started with number 0.
        // We'll not decrease further.
        if (presentNumber > 0) {
            setMyNumber(this.currentNumber - 1);
            System.out.println(presentNumber + " - 1 =" +
                                this.currentNumber);
            System.out.println(" The number is decremented.");
        } else {
            System.out.println("Nothing more to decrement.");
        }
    }
}
```

The AdditionCommand class is a concrete Command class that is used to target NumberGenerator. Notice that it stores the receiver (NumberGenerator) as an instance variable. Here is the AdditionCommand class :

```
class AdditionCommand implements Command {
    private NumberGenerator numberGenerator;
    public AdditionCommand(NumberGenerator receiver){
        this.numberGenerator = receiver;
    }
}
```

```

@Override
public void executeCommand() {
    numberGenerator.incrementNumber();
}
@Override
public void undoPreviousCommand() {
    numberGenerator.decrementNumber();
}
}

```

In the same way, you can make another class, say `SubtractionCommand` if you want. Since I do not use such a class in this implementation, I leave this exercise for you. The remaining code is easy to understand, so go through demonstration 2 now.

Class Diagram

Figure 19-3 shows the new class diagram. To make it short, I don't show the client in this diagram.

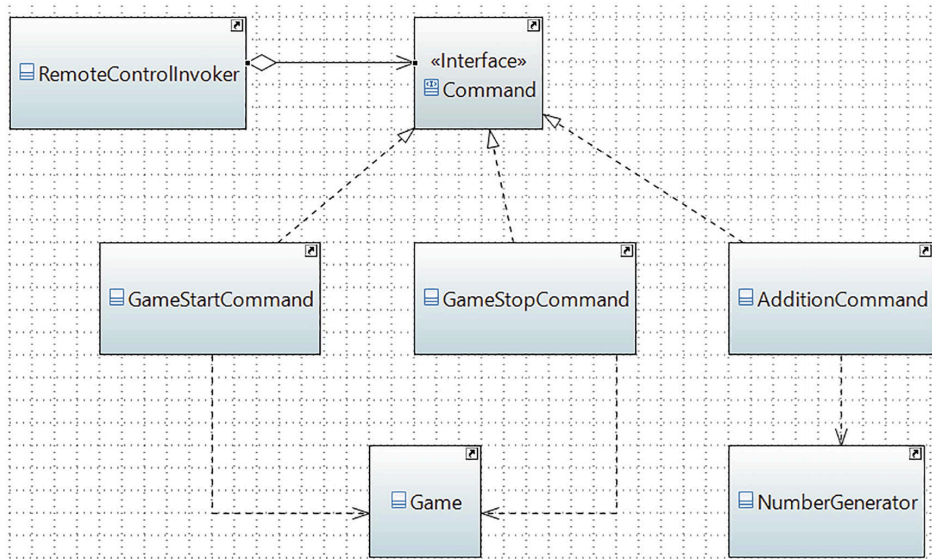


Figure 19-3 Class diagram showing the key participants in demonstration 2

Demonstration 2

Here is the complete program. All participants are stored inside a new package, `jdp3e.command.implementation_2`.

```

import java.util.ArrayList;
// Receiver- 1(Game.java)
class Game {
    String gameName;
    public Game(String name) {
        this.gameName = name;
    }
    public void start() {

```

```

        System.out.println(gameName + " is on.");
    }
    public void displayScore() {
        System.out.println("The score is changing from time to time.");
    }
    public void stop() {
        System.out.println("The game of " + gameName + " is stopped.");
    }
}
// Receiver-2(NumberGenerator.java)
class NumberGenerator {
    private int currentNumber;
    public NumberGenerator() {
        currentNumber = 0;
        System.out.println("The initial number is: " + currentNumber);
        System.out.println("You can increase it, but you
                            cannot decrease it further.");
    }
    public int getMyNumber() {
        return currentNumber;
    }
    public void setMyNumber(int currentNumber) {
        this.currentNumber = currentNumber;
    }
    public void incrementNumber() {
        System.out.println(" Received an increment number request.");
        int presentNumber=getMyNumber();
        setMyNumber(currentNumber + 1);
        System.out.println(presentNumber + " + 1 = " + getMyNumber());
    }
    public void decrementNumber() {
        System.out.println(" Received a decrement number request.");
        int presentNumber = this.currentNumber;
        // We started with number 0.
        // We'll not decrease further.
        if (presentNumber > 0) {
            setMyNumber(this.currentNumber - 1);
            System.out.println(presentNumber + " - 1 =" +
                               this.currentNumber);
            System.out.println(" The number is decremented.");
        } else {
            System.out.println("Nothing more to decrement...");
        }
    }
}
// Command.java
interface Command {
    // To execute a command
    void executeCommand();
    // To undo last command execution
    void undoPreviousCommand();
}

```

```
// GameStartCommand.java
class GameStartCommand implements Command {
    private Game game;
    public GameStartCommand(Game game) {
        this.game = game;
    }
    @Override
    public void executeCommand() {
        System.out.println("Game is about to start.");
        game.start();
        game.displayScore();
    }
    @Override
    public void undoPreviousCommand() {
        System.out.println("Undoing the start command.");
        game.stop();
    }
}

// GameStopCommand.java
class GameStopCommand implements Command {
    private Game game;
    public GameStopCommand(Game game) {
        this.game = game;
    }
    @Override
    public void executeCommand() {
        System.out.println("Finishing the game.");
        game.stop();
    }
    @Override
    public void undoPreviousCommand() {
        System.out.println("Undoing the stop command.");
        game.start();
        game.displayScore();
    }
}

// AdditionCommand.java
class AdditionCommand implements Command {
    private NumberGenerator numberGenerator;
    public AdditionCommand(NumberGenerator receiver){
        this.numberGenerator = receiver;
    }
    @Override
    public void executeCommand() {
        numberGenerator.incrementNumber();
    }
    @Override
    public void undoPreviousCommand() {
        numberGenerator.decrementNumber();
    }
}

// RemoteControlInvoker.java
```

```

import java.util.ArrayList;
class RemoteControlInvoker {
    Command currentCommand, lastCommandPerformed;
    ArrayList<Command> savedCommands = new
                                ArrayList<Command>();
    public void setCommand(Command command) {
        this.currentCommand = command;
    }
    public void executeCommand() {
        currentCommand.executeCommand();
        savedCommands.add(currentCommand);
        lastCommandPerformed = currentCommand;
    }
    public void undoCommand() {
        // Undo the last command executed
        lastCommandPerformed.undoPreviousCommand();
        // Remove it from saved command list
        savedCommands.remove(lastCommandPerformed);
    }
    public void undoAllStoredCommands() {
        for (int i = savedCommands.size(); i > 0; i--) {
            // Get a restore point and undo previous commands
            (savedCommands.get(i - 1)).undoPreviousCommand();
        }
        // Removing everything from the saved list
        savedCommands.clear();
    }
}
// Client.java
class Client {
    public static void main(String[] args) {
        System.out.println("***Command Pattern Demonstration-2.***\n");
        // Client holds both the Invoker
        // and Command Objects.
        RemoteControlInvoker invoker = new
                                RemoteControlInvoker();
        System.out.println("-----Testing the game scenarios-----\n");
        // Initializing a receiver
        Game gameName = new Game("Golf");
        // Initializing commands to start and stop a game.
        GameStartCommand gameStartCommand = new
                                GameStartCommand(gameName);
        GameStopCommand gameStopCommand = new
                                GameStopCommand(gameName);
        System.out.println("Starting the game.");
        invoker.setCommand(gameStartCommand);
        invoker.executeCommand();
        System.out.println("\nIssuing a stop command now.");
        invoker.setCommand(gameStopCommand);
        invoker.executeCommand();
        // System.out.println("\nUndoing the previous command now.");
        // invoker.undoCommand();
    }
}

```

```

        System.out.println("\n---Testing the number scenarios---\n");
        // Initializing a receiver
        NumberGenerator numberGenerator = new
                                NumberGenerator();
        // Initializing commands to start and stop a game.
        AdditionCommand addNumberCommand = new
                                AdditionCommand(numberGenerator);
        invoker.setCommand(addNumberCommand);
        System.out.println("\nWe'll test a series of do/undo operations now.");
        System.out.println("Incrementing a number three times.");
        invoker.executeCommand();
        invoker.executeCommand();
        invoker.executeCommand();
        System.out.println("Undoing the last command only.");
        invoker.undoCommand();
        System.out.println("Calling the undoAllStoredCommands() now.");
        invoker.undoAllStoredCommands();
    }
}

```

Output

Here's the new output:

```

***Command Pattern Demonstration-2.***
-----Testing the game scenarios -----
Starting the game.
The game is about to start.
Golf is on.
The score is changing from time to time.
Issuing a stop command now.
Finishing the game.
The game of Golf is stopped.
-----Testing the number scenarios-----
The initial number is: 0
You can increase it, but you cannot decrease it further.
We'll test a series of do/undo operations now.**
Incrementing a number three times.
    Received an increment number request.
0 + 1 = 1
    Received an increment number request.
1 + 1 = 2
    Received an increment number request.
2 + 1 = 3
Undoing the last command only.
    Received a decrement number request.
3 - 1 =2

```

```
The number is decremented.  
Calling the undoAllStoredCommands() now.  
Received a decrement number request.  
2 - 1 =1  
The number is decremented.  
Received a decrement number request.  
1 - 1 =0  
The number is decremented.  
Undoing the stop command.  
Golf is on.  
The score is changing from time to time.  
Undoing the start command.  
The game of Golf is stopped
```

.

Previous chapter

< [18. Iterator Pattern](#)

Next chapter

[20. Memento Pattern](#) >